

Toward Optimal Broadcast Mode in Offline Finding Network

Tong Li , Member, IEEE, Yukuan Ding , Jiaxin Liang , Kai Zheng, Senior Member, IEEE, Xu Zhang , Tian Pan , Dan Wang, Senior Member, IEEE, and Ke Xu , Fellow, IEEE

Abstract—This paper proposes *ElastiCast*, a novel Bluetooth Low Energy (BLE) broadcast mode that reduces the neighbor discovery latency in offline finding networks (OFNs). *ElastiCast* adapts the broadcast mode of the lost devices to the scan modes of the finder devices, considering their diversity. We start with an overview of OFNs, followed by a detailed analysis of the issues and challenges of existing solutions, which motivates the design of *ElastiCast*. Then we provide *Blender*, a simulator that models the neighbor discovery behavior of different broadcasters and scanners. By adopting *Blender*, *ElastiCast* can be implemented with three components: Local Optima Estimation, Common Interest Extraction, and Interval Multiplexing, in which we capture the key features of BLE neighbor discovery and globally optimize the broadcast mode interacting with diverse scan modes. Experimental evaluation results and commercial product deployment experience demonstrate that *ElastiCast* is effective in achieving stable and bounded neighbor discovery latency within the power budget.

Index Terms—Offline finding network, neighbor discovery, Bluetooth low energy, scan mode diversity.

Received 29 December 2023; revised 5 November 2024; accepted 17 February 2025. Date of publication 25 February 2025; date of current version 5 June 2025. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62202473, Grant 62441230, and Grant 62372053, in part by the National Science Foundation for Distinguished Young Scholars of China under Grant 62425201, in part by the Science Fund for Creative Research Groups of the NSFC under Grant 62221003, the Key Program of the NSFC under Grant 61932016 and Grant 62132011. The work of Dan Wang was supported by RGC GRF under Grant 15200321, Grant 15201322, Grant 15230624, Grant ITC ITF-ITS/056/22MX, Grant ITF-ITS/056/22MX, and Grant ITS/052/23MX, and in part by the PolyU under Grant 1-CDKK and Grant G-SAC8. Recommended for acceptance by A. Capone. (Corresponding author: Jiaxin Liang.)

Tong Li is with DEKE Lab and Information School, Renmin University, Beijing 100872, China (e-mail: tong.li@ruc.edu.cn).

Yukuan Ding is with the Electrical Engineering, Mathematics and Computer Science (EEMCS), Delft University of Technology, 2600 AA Delft, The Netherlands (e-mail: y.ding-1@tudelft.nl).

Jiaxin Liang and Kai Zheng are with 2012 Labs, Huawei, Shenzhen 518129, China (e-mail: jiaxin.liang@huawei.com; kai.zheng@huawei.com).

Xu Zhang is with the School of Electronic Science and Engineering, Nanjing University, Nanjing 210023, China (e-mail: xzhang17@nju.edu.cn).

Tian Pan is with the School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: pan@bupt.edu.cn).

Dan Wang is with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong SAR PQ708, China (e-mail: dan.wang@polyu.edu.hk).

Ke Xu is with the Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: xuke@tsinghua.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TMC.2025.3545561>, provided by the authors.

Digital Object Identifier 10.1109/TMC.2025.3545561

I. INTRODUCTION

MOBILE devices, which are essential in our daily lives, might go missing at a time. Thus some wearable devices (e.g., My Buddy Tag, Moochies, Pocket Finder) are equipped with the feature of position tracking. These devices, however, require Internet access and are often costly and power-hungry, due to the complicated built-in communication components. To tackle this issue, some industrial pioneers (e.g., Tile, Nutspace and Nut Technology, Apple, and Samsung) have developed offline finding networks (OFN) that leverage nearby online devices (a.k.a., finder devices) to help users locate their lost devices even when the devices are offline. For instance, Apple has over 1 billion iPhones that already have the Find My application [1] installed. These iPhones have formed one of the largest crowd-sourced OFNs worldwide [2]. Apple's AirTag [3] is a tiny metal tracker that works with the Find My application, and can be attached to a keychain, dropped in a bag, or snapped onto luggage to keep track of these items. AirTag, together with its OFN service, is predicted as Apple's next billion-dollar business [4].

Nowadays, OFN remains in its infancy with plenty of unsettled issues. To the best of our knowledge, almost all prior works focus on privacy and security analysis of OFN systems [5], [6], [7], [8], [9]. Their goal falls into the category of enabling crowd search without leaking private data. This paper does not focus on the security and privacy issues of OFNs. Instead, we take a first step toward understanding the OFN framework (see Section III) and identify its design challenges from the perspective of performance. Particularly, in OFN applications, the user experience is closely related to the success ratio (probability) of finding the lost device, which is significantly affected by the neighbor discovery latency (see Section IV-A). Neighbor discovery is the prerequisite stage of OFN, in which a finder device tries to establish contact with the lost device in the BLE signal range. It involves interactions between broadcasters and scanners in a duty-cycling paradigm with three parameters: the broadcaster's broadcast interval (A), the scanner's scan window (W), and the scan interval (T).

Existing works typically focus on optimizing point-to-point BLE broadcasts and scans [10], [11], [12], [13], [14], [15], [16], [17], [18]. These prior works, however, fall into the category of setting broadcast mode in the case of homogeneous neighbor discovery, where all the scanners are with the same scan mode. Among them, Kindt et al. [18] have proposed the tight duty-cycle-dependent bounds on the worst-case discovery latency that

TABLE I
BATTERY LIFE WITH DIFFERENT BROADCAST INTERVALS

Battery	Capacity	A	Battery Life
ER14250	2400 mAh	500 ms	60 months
CR2032	240 mAh	1000 ms	9 months
CR2032	240 mAh	2000 ms	12 months

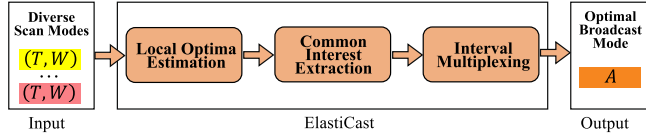


Fig. 1. Key modules in ElastiCast.

no prior neighbor discovery parameter setting approaches can beat, and concluded there is no further potential to improve the relationship between latency and duty-cycle in homogeneous neighbor discovery. In this paper, we argue that it is still possible to optimize discovery latency with a power budget in OFN due to the *scan mode diversity* of finder devices.

The finder devices are composed of a group of users that are close to and within the Bluetooth signal coverage of the lost device. From the perspective of the operators in the ecosystem of OFN, the broadcast mode of the lost device is usually controllable (i.e., private brand), while the scan modes of finder devices are uncontrollable. The users of finder devices are crowd-sourced and are probably in different power modes (e.g., power-saving mode and high-performance mode) or from different manufacturers (e.g., Samsung, Huawei, Oppo, Vivo, and Xiaomi from the Android ecosystem). Thus, the scan modes of the finder devices are diverse. The challenge is to find the optimal broadcast mode (pattern) that adapts to the diversity of the uncontrollable scan modes (see Section IV-D).

The broadcast mode affects both discovery latency and power consumption. On the one hand, a large broadcast interval may result in a large discovery latency, significantly reducing the possibility of finding lost devices (see Section IV-A). On the other hand, the power consumption by the broadcaster is inversely proportional to the broadcast interval. For example, with the default broadcast interval 2000 ms [19], the AirTag's CR2032 lithium coin battery life lasts for one year [3]. Still, it may be halved if the broadcast interval is reduced to 500 ms (see Table I). Hence, a smaller broadcast interval means higher power consumption, reducing the applicability of OFN systems.

It seems intuitive that choosing the minimal A within the power budget would maximize discovery. However, multiple broadcast intervals yield local minimum latency for a given W and T on the scanner (see Fig. 6). Therefore, choosing the minimal A may not always maximize discovery. On the other hand, the conventional method of neighbor discovery that adopts a fixed broadcast mode is far from satisfactory, as it may result in suboptimal performance of neighbor discovery in the case of scan mode diversity. In other words, its discovery latency is not bounded within the power budget (see Section IV-E). This paper presents a brand-new broadcast mode called *ElastiCast*. As shown in Fig. 1, *ElastiCast* deals with the scan mode diversity

issues through the following modules: Local Optima Estimation, Common Interest Extraction, and Interval Multiplexing.

Local Optima Estimation uses Blender (originally proposed in [20]), a neighbor discovery simulator that mimics the behavior of broadcasters and scanners. Differing from the conventional method of *random sampling*, Blender uses specifically designed models to generate a deterministic full distribution of latency (see Sections IV-B and VI-A). The input of estimation contains multiple settings of scan modes among all possible finder devices, and the output is the set of latency distributions as the functions of the broadcast interval.

Common Interest Extraction exploits the non-linear relationship between discovery latency and power consumption (see Section IV-C). It selects the common broadcast intervals that achieve minimized discovery latency among all scan modes. However, simply choosing a single optimal broadcast interval might introduce bias due to the random advertising delay (i.e., *adv_delay*), a pseudo-random value with a range of 0ms to 10ms generated for each broadcast event (see Section VI-B).

Interval Multiplexing overcomes the hurdles caused by *adv_delay*. It uses a mixture of multiple feasible broadcast intervals instead of a single one. It advances toward the global optima in the cases where *adv_delay* is non-negligible (see Section VI-C).

Experimental evaluation results show that *ElastiCast* is effective in realizing stable and low-latency BLE neighbor discovery within the power budget in the case of scan mode diversity. Specifically, compared to the conventional method of local optima, *ElastiCast* reduces the discovery latency by 50% to 90% within the power budget in our case studies. Our commercial product applying *ElastiCast* is also improves the success rate by over 11% compared to the state-of-the-art AirTag in a real-world deployment. This can be attributed to the elasticity of *ElastiCast*, which adapts well to the scan mode diversity by searching for the best broadcast pattern that achieves the globally minimized discovery latency within the power budget (see Section VIII).

The rest of this paper is organized as follows. Section II reviews the related work. Section III introduces the background and framework of OFN. Section IV explains the design rationale of *ElastiCast*. Section V summarizes the design of *ElastiCast* together with the Blender simulator, followed by the detailed procedure of *ElastiCast* in Section VI. We evaluate the performance of Blender in Section VII and the performance of *ElastiCast* in Section VIII, and conclude the paper in Section IX.

II. RELATED WORK

Offline Finding Networks: Prior studies on OFNs mainly focus on how to enable crowd search without leaking private data [5], [6], [7], [8]. For example, Apple proposed the asymmetric key-based way that encrypts location data with public keys [3], [8]. Mira Weller et al. [5] proposed a symmetric key-based way, called PrivateFind, to provide end-to-end encryption of location data. Other works have conducted an in-depth analysis of security and privacy issues based on these two ways. For example, Heinrich et al. [6] challenge OFN's security and privacy claims and examine the system design and implementation for

vulnerabilities through reverse engineering. Mayberry et al. [7] show that OFN's threat model for antitracking is dangerously incomplete and presents three strategies that a malicious tracker can use to evade detection by item safety alerts. Tonetto et al. [8] further present how OFNs can be used to estimate large groups of people, which is beyond its original purpose of tracking lost devices. In this paper, we do not focus on the privacy and security aspects of OFN systems. Instead, we take a first step towards understanding the OFN framework and highlight its design challenges from the perspective of neighbor discovery performance.

Neighbor Discovery: Efficient neighbor discovery aims to achieve the shortest possible discovery latency for a given power budget. To this end, a large number of broadcast and scan setting approaches [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31] have been proposed for general wireless neighbor discovery. For instance, following slotted neighbor discovery protocols where time is divided into fixed intervals that communication occurs within, Hello [27] identifies overlaps in active slots and offers a framework for abstracting parameter settings in active slot selections. Hedis [30] provides a better solution in asymmetric situations where the devices operate in different duty cycles. Diff-Codes [28] decreases worst-case and median latency by maximizing the likelihood of two devices being awake simultaneously. However, despite the necessity of synchronization between devices in slotted protocols, latency distribution is hardly observed in the optimization problem statements. Besides traditional slotted protocols, there are studies specifically designed for BLE neighbor discovery [10], [11], [12], [13], [14], [15], [16], [17], [18]. Examples include [10], [13], where realistic measurements are adopted as the basis for searching for the best parameter configurations. Meanwhile, [11] proposes the Effective-Scan-Window-based modeling for slotless BLE neighbor discovery, which, together with the subsequent works [12], [17], simulates and evaluates the discovery process. Kindt et al. [18] have proposed the tight duty-cycle-dependent bounds on the worst-case discovery latency and concluded there is no further potential to improve the relationship between latency and duty-cycle in homogeneous neighbor discovery. These prior works, however, fall into the category of setting broadcast mode in the case of homogeneous neighbor discovery, where all the scanners are with the same scan mode. To the best of our knowledge, this is the first work that captures the key features of OFN neighbor discovery where finder devices vary in different scan modes and overcome the challenges induced by scan mode diversity.

Discovery Latency Estimation: The correlation between parameter combinations of neighbor discovery, specifically BLE-like instances, and discovery latency is naturally complicated due to the flexibility of parameter selection, which arouses the requirement for calculating the discovery latency. The approaches adopt different methodologies including a pragmatic idea to conduct realistic measurements [10], [13]. To overcome the randomness and wall-clock time consumption in real-world experiments, more modeling-based methods are proposed, considering the limitation of theoretical analysis triggered by the complex interaction between the scanner and the broadcaster.

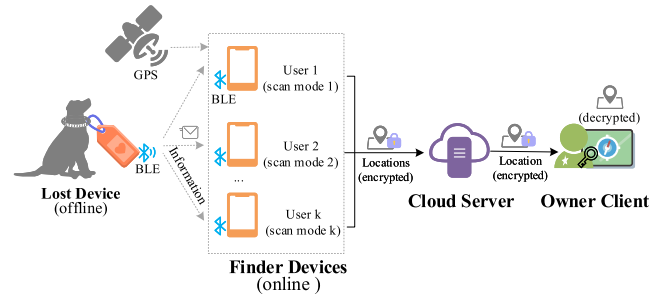
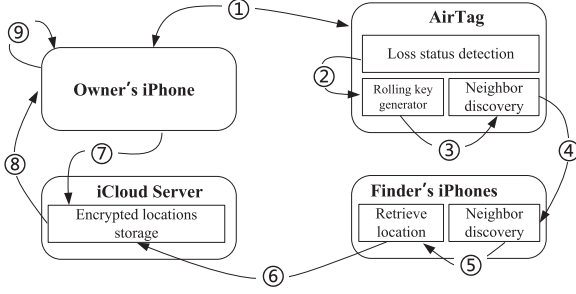


Fig. 2. Ecosystem overview of OFN.

The majority of recent modeling methods [15], [32], [33] utilize the *Chinese Remainder Theorem* (CRT) to calculate latency for every range-entrance situation. Besides, to either avoid redundant and inefficient calculations or reveal the impact of the parameter configuration on the discovery process, more analytical models have been proposed. For example, [34] builds the Partition model to group the offsets of the first scan window to the range-entrance event, and derives an average latency in terms of the number of broadcasting events before discovery. Unfortunately, the model is complicating the discovery analysis and is tailored for average and worst-case latency. Even if the model is reformed to produce the distribution of discovery latency, the fixed duty-cycle of the broadcaster that deeply entangles within the model and the derivation steps prevents the model to generalize to dynamic duty-cycle, including the broadcasting with random delay and the flexible broadcasting intervals. The Coverage Model from [18], though proposed earlier than many neighbor discovery models, manifests its conciseness in revealing the nature of the discovery process. It is utilized to formulate the minimum worst-case latency at a given fixed duty-cycle or with flexible intervals but an expected duty-cycle. Nevertheless, the Coverage Model was delivered with excessive focus on the configurations that achieve the minimum possible worst-case latency, neglecting the other configurations as well as their performance assessment outside of the worst-case scenario. In other words, the potential of Coverage Model has not been fully leveraged by neither the original nor the subsequent studies.

III. OFFLINE FINDING NETWORK: AN OVERVIEW

The ecosystem of OFN consists of four types of roles. As shown in Fig. 2, the *Lost Device* is usually a thin device (e.g., watch, headset, tag) without complicated built-in functions of communication and positioning. It can also be a rich device (e.g., phone, tablet) without online access. The *Finder Devices* are a group of users who are close to and within the Bluetooth signal coverage of the lost device. These devices are usually rich devices with built-in functions of communication and positioning, acting as volunteers to help the offline lost device report its location. The *Cloud Server* provides the storage service of the reported locations in the cloud. Since the location data is encrypted, the privacy of finder devices is protected. The *Owner Client* is usually a device with Internet access (e.g., phone, tablet) that decrypts the location data queried from the cloud server.



- Step 1: Generate public-private key pairs and bind AirTag to owner's iPhone
Step 2: Detect if the AirTag is lost. If yes, go to Step 3
Step 3: Generate rolling public keys periodically in minutes
Step 4: AirTag broadcasts public key as content via BLE and finder's iPhones recognize the key throughout neighbor discovery
Step 5: Retrieve current location and encrypt it with the broadcasted public key
Step 6: Upload the encrypted location record
Step 7: Generate the list of rolling public keys in the last days and query the iCloud server
Step 8: Return the encrypted locations records for the list of requested keys
Step 9: Decrypt location records with the private key and obtain an approximate location

Fig. 3. Basic framework of Apple's Find My.

To understand how OFNs work in detail, we take Apple's Find My feature that works with Apple's iPhone and AirTag as a case study. The basic framework of Apple's Find My is illustrated in Fig. 3. We can see that there are two additional components called *iCloud Server* and *Finder's iPhones* that help the *Owner's iPhone* find the lost *AirTag*.

For initialization, the owner binds AirTag to his/her iPhone, and the Find My application generates the Elliptic Curve key pair, whose public key is stored in the AirTag (*Step 1* in Fig. 3). When the AirTag is detected and marked as lost (in *Step 2*), it generates rolling public keys periodically (e.g., 10 minutes) using a shared secret [35] (in *Step 3*). The AirTag broadcasts the public keys periodically (i.e., broadcast interval $A = 2000$ ms), and multiple nearby iPhones recognize and get the broadcasted public key via BLE neighbor discovery (in *Step 4*). These finder's iPhones retrieve their current location, encrypt the location with the public key (in *Step 5*) and upload the encrypted location record (in *Step 6*). To find the lost AirTag, the paired owner's iPhone generates the list of the rolling public keys that the AirTag would have used in the last days and queries the iCloud server (in *Step 7*). The server returns the encrypted location records (in *Step 8*) for the list of requested public keys, and finally, the owner's iPhone decrypts the location records with its private key and obtains an approximate location of the lost AirTag (in *Step 9*).

We have also investigated other OFNs. Although some of them (e.g., PrivateFind) adopt a different key management scheme, the basic workflow remains similar. Thus we have found that Apple's framework is representative in terms of the four major components of an OFN as illustrated in Fig. 3, which is not surprising given it is a natural extension to the conventional finding networks with crowd-sourcing.

IV. NEIGHBOR DISCOVERY LATENCY OF OFN: ISSUES AND CHALLENGES

This section discusses several observations on the issues and challenges in the neighbor discovery of OFN.

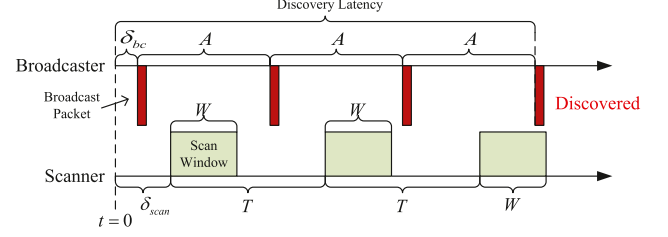


Fig. 4. The duty-cycling of BLE neighbor discovery. T , W , and A are the scan interval, scan window, and broadcast interval, respectively. δ_{bc} and δ_{scan} are the entrance time.

A. Neighbor Discovery Latency Matters in OFN

As the prerequisite stage of OFN, neighbor discovery is an essential process where a finder device seeks to first contact the lost device in the BLE radio range (i.e., *Step 4* in Fig. 3). Generally, neighbor discovery involves the interactions between a broadcaster and a scanner, where the broadcaster broadcasts signals periodically and the scanner operates in a duty-cycling paradigm (see Fig. 4). Neighbor discovery succeeds at the time when a scanner captures the complete packet in a broadcast event, which should occur immediately when the scanner device enters the radio range of the broadcaster device if the devices are scanning/broadcasting continuously. The discovery latency is measured from the time when both devices enter the range of reception. We refer to this outset as $t = t_0 = 0$ as in Fig. 4, denoted by the entrance time or range-entrance event. From the range-entrance event, a broadcasting event for a packet to be broadcasted should occur after a period of δ_{bc} ; a scanning window should occur after a period of δ_{scan} . Thereby, the discovery latency is defined as:

$$L = \delta_{bc} + \mathcal{N} \cdot A \quad (1)$$

where the $(\mathcal{N} - 1)$ -th broadcast packet is the first one, after the entrance time, overlapping within an active scan window. Specifically, a broadcast at time $t = \delta_{bc} + \mathcal{N} \cdot A$ will overlap with an active scan window if the following condition is satisfied:

$$(\delta_{bc} - \delta_{scan} + \mathcal{N} \cdot A) \bmod T < W$$

The modulo operation accounts for the periodic nature of the scanning schedule, effectively mapping the time difference into a single scanning interval. Therefore, $\mathcal{N}$ can formally defined as the minimal non-negative integer achieving the overlap:

$$\mathcal{N} = \min \left\{ n \in \mathbb{Z}_{\geq 0} \mid (\delta_{bc} - \delta_{scan} + n \cdot A) \bmod T < W \right\}$$

As illustrated in Fig. 4, the discovery latency is related to the broadcaster's broadcast interval (A), the scanner's scan window (W), and scan interval (T), where the scan duty cycle is computed by $D = \frac{W}{T}$. In general, power consumption is proportional to D and is inversely proportional to A , while a larger A or a lower D leads to an interleaved activity pattern between the broadcaster and the scanner, which may induce unacceptably large discovery latency [18], [36]. Since both the lost devices and the finder devices in OFN are power-sensitive [3], [37], the large discovery latency may significantly reduce the possibility of finding lost devices.

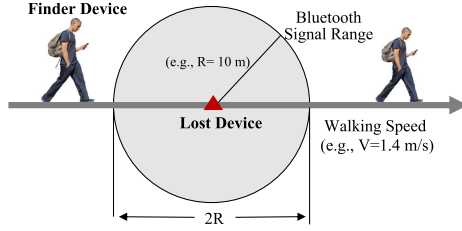


Fig. 5. Neighbor discovery between finder device and lost device in the walking scenario.

To explain this more clearly, we give an example of how a lost device is found by a finder device. As shown in Fig. 5, a person with a finder device passes the Bluetooth signal range (e.g., a cycle area with a radius of $R = 10$ meters [38]) of a lost device at a walking speed (e.g., $V = 1.4$ meters per second).¹ Assuming the ideal case where a person walks along the diameter of the circle, the neighbor discovery latency should not exceed the time the person spends within the Bluetooth signal range (i.e., latency tolerance, $\frac{2R}{V} \approx 14$ seconds). Otherwise, the finder device fails to find the lost device, and the opportunity is wasted. Thus we conclude that neighbor discovery latency matters in OFN concerning the possibility of finding lost devices.

B. A Full Latency Distribution Requires Consideration

In previous parameter evaluation tools for BLE-like neighbor discovery, the rubric focuses on the mean or the worst-case latency (i.e., the longest discovery latency under a parameter setting) [18]. However, a metric equipped with probabilistic features can be required. In some services such as proximity tracing [40], [41] and assets tracking [5], [42], the accomplishment of discovery becomes an expectation rather than a necessity due to the restriction of the length of time reserved for discovery.

Specifically, there is an expected rate $P\%$ of successful discovery within a given time L , where good parameters here are supposed to reach this expectation. This quantified requirement is suitable to be represented through a CDF. In a latency CDF, each probability value P corresponds to a latency value L_P , which is the P -percentile latency. Here, reaching a success rate of no lower than $P\%$ in L time has an equivalent expression: the P -percentile latency being smaller than L . In order to provide any P -percentile latency, a full distribution of latency values is hence necessary. The importance and configuration of P is revisited in Sections V-B and VI-B.

While a series of previous works are mentioned to possess the functionality to generate full latency distribution, such distributions stem from an incomplete domain. As we claimed in Section IV-A, the first scan window after the range-entrance event should also have a time offset δ_s from the range-entrance event rather than stick at time zero. While the previous distributions were generated upon a domain with a fixed first

¹ A person's state of motion contains static scenario (e.g., still), low-speed scenario (e.g., walking), and high-speed scenario (e.g., running or riding vehicles). Since the static scenario can hardly suffer from performance issues, and the high-speed scenario is regarded as a rare case in OFN, in this paper we mainly focus on the most representative case of walking.

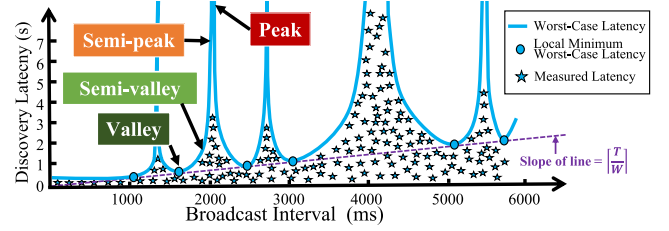


Fig. 6. The distribution of the worst-case discovery latency (upper bound). $W = 1024$ ms and $T = 4096$ ms (i.e., a representative scan mode labeled as BALANCED in modern Android systems [39]).

scan window, the refined range-entrance event will significantly enlarge the domain and can produce a more practical distribution. The refinement, however, cannot be directly applied to the existing methods. Since the simulation time for both CRT-based and effective-scan-window-based methods can be significantly prolonged due to their need to traverse the entire domain, this becomes impractical when a large number of parameter configurations need to be evaluated.

C. Discovery Latency is a Non-Linear Function of Power Consumption

It has been well-studied that the trade-off between discovery latency and power consumption should be carefully handled to meet the application's required performance under the power constraint. Intuitively, discovery latency is inversely proportional to power consumption. However, this is only true when $A \leq W$, which is well studied in the prior studies [11], [43]. Recently, Kindt et al. [16] have exposed the non-linear relationship between discovery latency and power consumption when $A > W$. As shown in Fig. 6, given a certain scan mode, i.e., scan window (W) and scan interval (T), there exists an upper bound of discovery latency in the worst case for each broadcast interval (A). As plotted in the blue line, the worst-case latency is a non-linear function of A .

Since both broadcaster and scanner may randomly come into the range of reception, i.e., the range-entrance times δ_{bc} and δ_{scan} (see Fig. 4) are the stochastic factors that decide the actual measured discovery latency. As a result, for a given A , the measured discovery latencies (plotted as blue stars) range from zero to the upper bound (i.e., worst-case latency).

In Fig. 6, we further find that there exist multiple local minimum worst-case latencies (plotted as blue cycles). It is demonstrated in the prior work [18] that all the local minimum worst-case latencies follow a line with a slope of $\lceil \frac{T}{W} \rceil$. This reveals that the distribution of the worst-case latency varies with the scan mode. For the sake of description, we define that A is within "valley area" when A achieves the local minimum worst-case latency, and A is within the "semi-valley area", the "semi-peak area", and the "peak area", respectively, according to the increased latency compared to the corresponding local minimum worst-case latency.

To summarize, the existence of multiple valley areas implies that we might fail to reduce discovery latency by simply reducing

A. Instead, we need to “smartly” select A within valley areas while not violating the power constraint.

D. Finder Devices Vary in Scan Modes

As shown in Fig. 2, the finder devices are composed of a group of crowd-sourced users that are nearby and under the Bluetooth signal coverage of the lost device. Hence the scan modes of finder devices are usually uncontrollable. As a result, these finder devices usually show diversity in scan modes. We further summarize the causes, i.e., dynamics and customization, of this diversity below.

Dynamics: By default, the Android Open Source Project (AOSP) supports three scan modes labeled as LOW_LATENCY, BALANCED, and LOW_POWER [39]. For a certain finder device, the scan mode might depend on whether the device is in power-saving mode or high-performance mode, whether the device screen is on or off, and whether the application is running in the foreground or background. For example, Android recommends applying LOW_LATENCY scan mode with a duty cycle of 100% only when the application is in the foreground, and LOW_POWER scan mode with a duty cycle of 10% when the application is in the background.

Customization: Some manufacturers might customize the scan mode in their products to achieve a better trade-off between scan frequency and power consumption. For example, iOS proposes a customized scan mode that scans 30 ms for every 300 ms in its OFN (i.e., Apple’s Find My [3]), while HarmonyOS recommends a scan mode that scans 20 ms for every 600 ms in its HiLink protocols [44].

It is worth noting that forming a global finder network with different brands could help alleviate performance challenges brought about by scan mode diversity. However, both technical and non-technical issues arise. On the technical side, this approach cannot fully eliminate the diversity in scan modes. This is because even for phones of the same brand, the scanning mode may differ depending on whether the screen is on or off. On the non-technical side, each phone manufacturer aims to optimize the performance of its devices in specific scenarios, leading to the customization of their scan modes.

E. Scan Mode Diversity Results in Local Optima

As discussed above, for each scan mode we should carefully search an optimal broadcast interval to obtain the minimum upper bound of discovery latency. However, the distribution of the worst-case latency varies with the scan mode. As a result, an optimal broadcast interval for a certain scan mode might not be the optimal broadcast interval for another scan mode.

To explain this more clearly, in Fig. 7 we give an example of neighbor discovery in the case of multiple scan modes. Specifically, three types of finder devices act as scanners, and the lost device acts as the broadcaster. Their scan modes are 30 ms/300 ms, 60 ms/600 ms, and 60 ms/3000 ms, respectively. As shown in Fig. 7(a), when we set the broadcast interval $A = 2000$ ms according to the default settings in modern commercial products (e.g., AirTag), we find that $A = 2000$ ms is within the valley area of the scan mode 1, but $A = 2000$ ms is within the peak

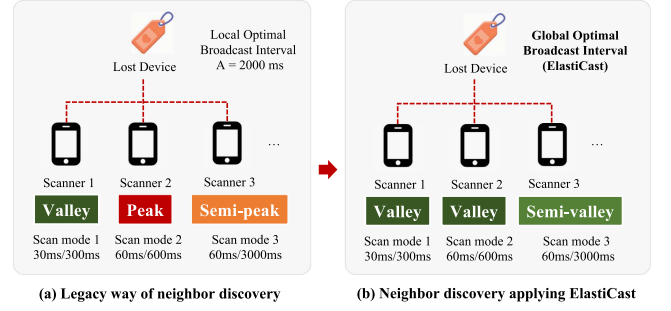


Fig. 7. ElastiCast aims to change from local optima to global optima.

and semi-peak area of the scan modes 2 and 3, respectively. This reveals that the scan mode diversity results in local optima.

Based on these observations, the legacy way of neighbor discovery is far from satisfactory. In this paper, we aim to search for the global optimal broadcast pattern that makes the broadcast interval(s) within the valley or semi-valley area for all types of scan modes (see Fig. 7(b)). Thus we proposed ElastiCast as we will elaborate next.

V. ELASTICAST OVERVIEW

The primary goal of ElastiCast is to achieve global optima by overcoming the challenges induced by scan mode diversity. In this section, we first formalize the problem of neighbor discovery with scan mode diversity, and then we introduce the framework of ElastiCast.

A. Problem Formalization

We model the overall performance of discovery latency by introducing the metric, *weighted average discovery latency*, denoted as $\hat{l}$. Given a set $\mathbf{S} = \{s_1, s_2, \dots, s_n\}$ including n types of scan modes, where s_i is the i^{th} scan mode. Let ω_i and l_i be the market share and the discovery latency of the i^{th} scan mode, respectively. Here the market share represents the percentage of different types/states of finder devices in the market, which approximates the percentage of states of finder devices near a lost device. Then the weighted average discovery latency $\hat{l}$ is computed as follows

$$\hat{l} = \sum_{i=1}^n \omega_i \cdot l_i \quad (2)$$

As mentioned in the previous section, the duty cycle of a lost device cannot be vast due to power constraints. ElastiCast aims to minimize the weighted average discovery latency within the power budget. In other words, we aim to achieve

$$Z = \min \hat{l}, \text{ s.t. } A \geq A_{\min} \quad (3)$$

where A is the selected broadcast interval, and $A_{\min}$ is the minimum broadcast interval. As illustrated in Table I, the broadcaster’s power consumption (battery life) increases proportionally with the broadcast interval. Hence, $A_{\min}$ can be equivalently regarded as the power budget of a broadcaster in this paper.

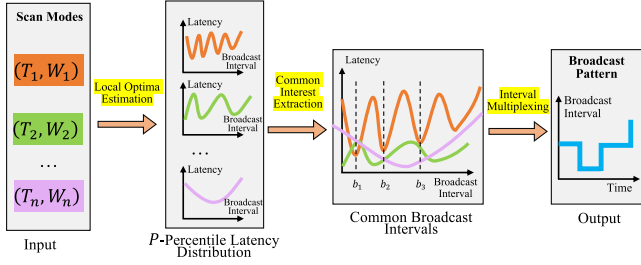


Fig. 8. The basic framework of data flow in ElastiCast.

It is worth noting that market share changes can affect the optimality of the undecided broadcast interval parameters in ElastiCast. This paper assumes that market share changes, calculated at a regional level, are relatively slow within most users' activity areas. However, as future work, we plan to offer a dedicated app, which will run on smartphones and connect to the TAG to configure its broadcast mode based on the dynamic market share of scanning modes.

B. The ElastiCast Framework

The framework of ElastiCast is displayed as a data flow diagram in Fig. 8, which contains the input/output and has three intermediate modules primarily. In this section, we briefly introduce the design rationale of each module as below.

Input and Output: When applying ElastiCast in the neighbor discovery of OFN, the input contains multiple settings of scan modes among all possible finder devices. The output is the broadcast pattern of the lost device by multiplexing the feasible broadcast intervals.

Local Optima Estimation: For each scan mode, given a range of the broadcast intervals, the goal of this module is to generate the P -percentile ($P \in [0, 100]$) discovery latency from the latency distribution.² A straightforward way is to repeatedly conduct real-world experiments [10], [13] which have low efficiency and a high cost, much less the bias induced by wireless interference [45]. Another way is building a mathematical model that provides a function to obtain a deterministic discovery latency from a certain set of parameters. However, the state-of-the-art modeling work [18] only provides the bound of discovery latency, while the modeling of the distribution of all discovery latency values is still an open issue in this field. Controllable and reproducible, as shown in Fig. 9, we build a lightweight neighbor discovery simulation tool, Blender [20], that simulates the behavior of the broadcaster and scanner according to the parameter configurations. Blender distinguishes itself from the legacy way of random sampling by applying equivalence-relation-based *Case Projection* according to the *Base-Case Simulation*, this not only avoids the costly brute-force traversal through all cases but also achieves more deterministic results than that of the random sampling.

²By default, $P = 100$ refers to the worst-case discovery latency. However, in practical cases manufacturers may care more about a range of tail latencies (e.g., 95-percentile), thus we leave P as the customizable parameter.

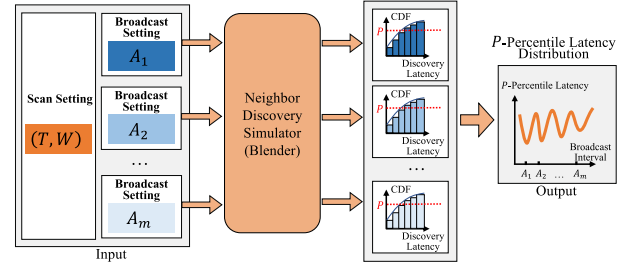


Fig. 9. The work progress of local optima estimation.

Common Interest Extraction: For each scan mode, the broadcast intervals near the valley or semi-valley area are defined as the *interest* of this scan mode. As shown in Fig. 8, the latency distributions vary with different types of scan modes. This module extracts the common interest by finding the common broadcast intervals (e.g., b_1 , b_2 , and b_3 in Fig. 8) that achieve the locally minimized weight average discovery latency (i.e., (2)) among all types of scan modes. This serves as a basis for global optimization of neighbor discovery performance in the case of scan mode diversity.

Interval Multiplexing: A naive way of ElastiCast is to apply a constant broadcast mode where a single broadcast interval is carefully chosen through the common interest extraction. However, as specified in the BLE standards [46], the interval between two consecutive broadcast events is not a constant but is mandatorily longer than the settled broadcast interval by a random period within 10 ms, called the random advertising delay (denoted by adv_delay , and $adv_delay \in [0, 10]$ ms). As a result, a large adv_delay might induce a negative effect for the naive way of choosing a single broadcast interval. Specifically, a broadcast interval within the valley area might be shifted to a non-valley area due to the mandatory and random adv_delay (see Section VI-B for more details). To tackle this issue, the interval multiplexing module steps further toward the global optima by dealing with adv_delay . The design rationale is that different broadcast intervals show different sensitivity to the adv_delay (see Fig. 15), and multiplexing the complementary broadcast intervals might compensate for the negative effect induced by the adv_delay . Specifically, we apply the broadcast pattern selected from the *Single Broadcast Pattern* and the *Alternation Broadcast Pattern*. The decision-making of the broadcast pattern and the corresponding parameter settings are made according to the minimized weighted average discovery latency for a given power budget.

VI. DESIGN DETAILS

In this section, we proceed to the design details of the specific modules in ElastiCast. Notations are listed in Table II.

A. Estimate the Local Optima

The parameter selection scheme of ElastiCast relies on the awareness of P -percentile latency under the candidate broadcasting and scanning configurations, as discussed in Section V-B. In this section, we first present a method along with

TABLE II
NOTATIONS

Name	Description
A	Broadcast Interval
T	Scan Interval
W	Duration of a scan window
δ_{bc}	Offset between broadcast and scan sequences
$\mathcal{A}_i$	The i -th broadcasting event in a given broadcasting sequence
$\mathcal{G}$	A time segment to be <i>covered</i>
ν	Overlap length between newly covered and previously covered periods in $\mathcal{G}$
Υ_i	Intermediate discovery latency produced by broadcasting event $\mathcal{A}_i$
Υ_f	Latency from the first broadcasting event to full coverage of segment $\mathcal{G}$
$P_l(\Upsilon_i)$	Collaborative probability of achieving intermediate latency Υ_i with $\delta_{bc} = 0$
$\mathcal{L}_\Upsilon$	List of latency-probability pairs $(\Upsilon_i, P_l(\Upsilon_i))$
ε	Deadline latency beyond which it is considered insignificant and all represented by ε
$\mathcal{Q}$	Root sequence of broadcasting intervals to be repeated with dynamic broadcast interval
$\mathcal{A}_j^{\mathcal{Q}}$	j -th broadcasting event in $\mathcal{Q}$ selected as the first broadcasting event after range entrance
τ_j	Interval between events $\mathcal{A}_j^{\mathcal{Q}}$ and $\mathcal{A}_{j-1}^{\mathcal{Q}}$ in $\mathcal{Q}$

its underlying principles illustrated, to produce the P -percentile latency under given broadcasting and scanning configuration. We then introduce the architecture of the Blender simulator built upon the aforementioned P -percentile latency generation scheme.

1) *Beyond Worst-Case Latency: The Latency Probability Distribution:* To know the P -percentile latency for any P , it is necessary to retrieve the latency's probability distribution, over the stochastic phase difference between the sequences of broadcasting events/scanning windows. While neighbor discovery involves the interaction between two devices, an intuitive idea for producing the distribution is to reproduce the discovery process from a global perspective; inefficiently sampling from all possible phase difference values nevertheless can produce the latencies from start to timestamps that both devices are simultaneously awake. But furthermore with sampling, neither we achieved a deterministic distribution nor we understood the principles of how the broadcasting/scanning parameters are alternating the distribution.

Exclusions in the discovery process: Rather than the random sampling method, we can bring benefit in determinism of the distribution and the estimation efficiency through shifting the

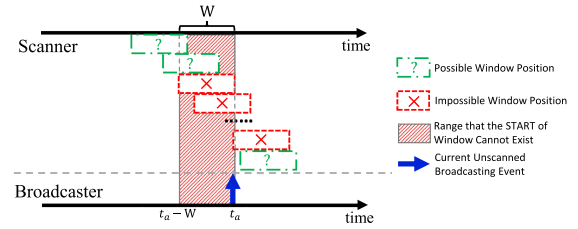


Fig. 10. The exclusion of possible scan window positions with an uncaptured broadcast package.

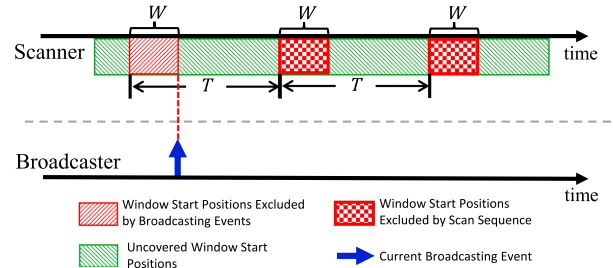


Fig. 11. Coverage of possible scan window positions by the single broadcasting event.

perspective to the broadcaster. The gist of a broadcaster's point of view is to conjecture about the relative time position of the scan sequence to the broadcast sequence. In other words, we build a probabilistic distribution over the time offsets of all possible scan sequences to the given broadcasting sequence. Fig. 10 illustrate the basic principle with assuming a broadcasting event occurring at time t_a . If it were captured by the scanner, a scan window must exist and envelop this broadcasting event. Conversely, it can be inferred from an uncaptured broadcasting event that there is a period of time where the scan window is certainly deactivated if no environmental interference occurs; if a broadcasting event occurs at t_a is not captured, the probability of any scan window to commence in the period $(t_a - W, t_a]$ is *excluded*. Noticeably, this exclusion of $(t_a - W, t_a]$ is not the end: the periodically repeating nature of the scan window also excludes $(t_a - W + i \cdot T, t_a + i \cdot T], i \in \mathbb{Z}^+$, as shown in Fig. 11. Therefore, as we traverse the broadcasting events, an increasing number of periods are excluded from being considered as starting timestamps for the scan windows, which we subsequently refer to as the *scan window's position*.

Coverage: Bridging Exclusion to Latency: How is the exclusion related to latency? We first consider the worst-case latency. To illustrate, we split the wall-clock time into identical segments $\mathcal{G}_i$, in other words, copies of a single segment $\mathcal{G}$, of length T . As shown in Fig. 12, after range-entrance, a broadcasting event (yellow arrow with circled 1) occurs. As discussed in the last paragraph, if discovered is not accomplished with current broadcasting event, we can infer the exclusion of scan window’s positions: a period of length W repeating in each segment. Notice that it is possible for this very first broadcasting event to achieve a discovery, it does not represent the worst case. As more broadcasting events occur, a new period will be excluded as long as discovery does not occur with the event; the segment will

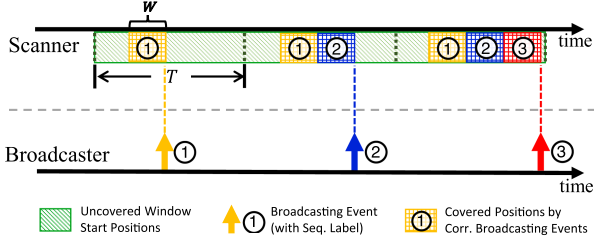


Fig. 12. Coverage of possible scan window positions by multiple broadcasting events.

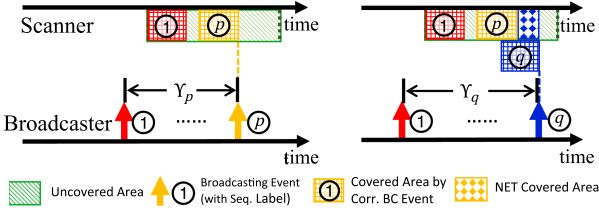


Fig. 13. Intermediate latency Υ_i w/ (left) and w/o (right) redundant coverage.

be gradually covered by the clusters of those excluded periods. Since the positions in $\mathcal{G}$ actually represent all different offsets between the scan windows sequence and the broadcasting events sequence, the longest time from the first broadcasting event to the discovery will be the time to exclude all possibilities for the offset, i.e., to cover $\mathcal{G}$. Conclusively, we formulate the Coverage Theory:

Theorem 6.1: Given the parameter set A, T, W , an infinite sequence of broadcasting events is used to generate clusters of impossible scan windows' positions. If the broadcasting events can fully cover the time-varying segment of length T within $\varrho = LCM(A, T)$ time (LCM stands for *Lowest Common Multiple*), there is a finite worst-case discovery latency, which is $A + \Upsilon_f$. A is the longest time from range-entrance to the first broadcasting event, and Υ_f is the time from the first broadcasting event to the broadcasting event that generates the last period to cover $\mathcal{G}$.

Expanded Coverage Theory for Full Latency Distribution: Theorem 6.1 was first presented in [18], unfortunately obscuring its underlying probabilistic nature in explanation: the gradual coverage of segment $\mathcal{G}$ is a process constructing the probability distribution of the scan sequence's offset to the broadcasting sequence. Therefore, Theorem 6.1 was not leveraged to generate the probability distribution of latency but only the worst-case counterpart. We propose the expansion of the Coverage Theory, illustrating how the latency distribution is produced respectively for the broadcaster with fixed and dynamic broadcast interval.

Latency Distribution for Fixed Broadcast Interval: With an overly focus on the worst-case latency, we considered the scenario that no discovery occurs during the coverage process of Segment $\mathcal{G}$. However, whenever a broadcasting event can contribute to a newly covered period in $\mathcal{G}$, it is possible to encounter an active scan window; the probability remains for some scan sequence offsets, in which a scan window starts within W before this broadcasting event. Consider a broadcasting event $\mathcal{A}_p$ in the left part of Fig. 13, where it can be inferred that: the probability of a scan window to start in this period of length W covered by $\mathcal{A}_p$ is

$\frac{W}{T}$. Therefore, an intermediate discovery latency, valued by Υ_m , is obtained as the time from the first broadcasting event $\mathcal{A}_0$ to $\mathcal{A}_m$, accompanied by a successful discovery with $\mathcal{A}_p$. Under the assumption $\delta_{bc} = 0$ (i.e., $P(\delta_{bc} = 0) = 1$), the probability for a scan window to overlap with $\mathcal{A}_p$ or the intermediate discovery latency to be Υ_p is:

$$P(\Upsilon_p) = P(\Upsilon_p | \delta_{bc} = 0) \cdot P(\delta_{bc} = 0) = \frac{W}{T} \quad (4)$$

Noticeably, any subsequent $\Upsilon_i (i \geq m)$ exists with the premise that no discovery occurs with $\mathcal{A}_j (j \leq m)$. Therefore, $\mathcal{A}_p$ still needs to produce a coverage to represent the case of an absent discovery. Each time a new period in $\mathcal{G}$ is to be covered, such as that produced by $\mathcal{A}_q$ in Fig. 13, it may collide with a previously covered period. The probability for Υ_q then becomes $P(\Upsilon_q) = P(\text{Window Start} \in \mathcal{G}_{[n]} \setminus \mathcal{G}_{[prev]})$, with $\mathcal{G}_{[n]}$ as the new period to cover with $\mathcal{A}_q$ and $\mathcal{G}_{[prev]}$ as the previously covered periods. Specifically, $P(\Upsilon_q)$ is calculated as $\frac{\max(W - \nu, 0)}{T}$ where ν is the length of $\mathcal{G}_q \cap \mathcal{G}_{prev}$ and $\max(W - \nu, 0)$ is the length of *net* covered period $\mathcal{G}_q \setminus (\mathcal{G}_q \cap \mathcal{G}_{prev})$. The intersected period is not considered since if a scan window starts in that area, the discovery should have been completed with a former broadcasting event.

With a list of broadcasting events $\{\mathcal{A}_i | i \in \mathbb{Z}\}$, the simulation will end up with a list $\mathcal{L}_\Upsilon$ of $(\Upsilon_i, P(\Upsilon_i))$ pairs. Besides, a deadline ε should be settled to represent all large and infinite latency, which are considered insignificant in the simulation result. Practically, an entry $(\varepsilon, 1 - \sum_i P(\Upsilon_i))$ is finally appended to $\mathcal{L}_\Upsilon$. This finite list will be expanded to a distribution over different range-entrance situations, i.e., $\delta_{bc} \in [0, A)$, while the generated list of pairs only describes the case $\delta_{bc} = 0$. Therefore, *case projection* becomes the process to involve all $\delta_{bc} \in [0, A)$ and produce the full latency distribution. Then the probability density function for a discovery latency L is formulated as:

$$P(L) = P(\delta_{bc} = \delta_{bc}^i) \cdot P(L - \delta_{bc}^i) = \frac{1}{A} \cdot P(L - \delta_{bc}^i), \quad (5)$$

where $\delta_{bc}^i = L - \Upsilon_i$, $\Upsilon_i \in \mathcal{L}_\Upsilon \wedge \Upsilon_i \leq L \wedge \Upsilon_i > L - A$

Latency Distribution for Dynamic Broadcast Interval: When fixed broadcast interval is applied, the broadcasting events are identical in terms of their relative positions within the sequence. Consequently, the coverage process of $\mathcal{G}$ is unaffected by which specific event in the sequence occurs first. However, with a flexibly duty-cycling broadcaster, this changes; as the broadcasting intervals vary, the interval between $\mathcal{A}_0$ and $\mathcal{A}_i$ also changes, leading to different values of Υ_i . A chance to recover this breakdown is that a determined broadcasting sequence in production scenarios should usually be recurrent, repeating a short sub-sequence stored in the limited ROM in Bluetooth modules. Therefore, a determined broadcasting sequence can be represented by a finite *root sequence* $\mathcal{Q}$, which records the intervals τ_j between a broadcasting event $\mathcal{A}_j^{\mathcal{Q}}$ and $\mathcal{A}_{j-1}^{\mathcal{Q}}$.

Stemming from the finite length of root sequence, the first broadcasting event after range-entrance can be enumerated from the root sequence, but with a probability $P(\mathcal{A}_j^{\mathcal{Q}}) = \frac{\tau_j}{\sum_k \tau_k}$ assigned. Similar to the situation of fixed broadcast interval, $\mathcal{L}_\Upsilon^j$ containing the $(\Upsilon_{j,i}, P(\Upsilon_{j,i} | \mathcal{A}_j^{\mathcal{Q}}))$ pairs is generated for each

$\mathcal{A}_j^Q$. Case projection is again conducted for each $\mathcal{L}_T^j$ in the same way in (5):

$$\begin{aligned} P(L|\mathcal{A}_j^Q) &= \sum_i P(\delta_{bc,j} = \delta_{bc,j}^i) \cdot P_l(L - \delta_{bc,j}^i | \mathcal{A}_j^Q) \\ &= \sum_i \frac{P(L - \delta_{bc,j}^i | \mathcal{A}_j^Q)}{\tau_j}, \end{aligned} \quad (6)$$

where we have:

$$\begin{aligned} \delta_{bc,j} &\in [0, \tau_j), \\ \delta_{bc,j}^i &= L - \Upsilon_{j,i}, \end{aligned}$$

$$\Upsilon_{j,i} \in \{\Upsilon_{j,u} \mid \Upsilon_{j,u} \in \mathcal{L}_T^j \wedge \Upsilon_{j,u} \leq L \wedge \Upsilon_{j,u} > L - \tau_j\}.$$

Note that different from (5), there can be multiple $\Upsilon_{j,i}$ possible to derive L after added by δ_{bc} . After marginalization, we derive the probability density function of a Latency L as:

$$P(L) = \sum_j P(L|\mathcal{A}_j^Q) \cdot P(\mathcal{A}_j^Q) = \sum_j \frac{\sum_i P_l(L - \delta_{bc,j}^i | \mathcal{A}_j^Q)}{\sum_k \tau_k}, \quad (7)$$

2) *The Architecture of Blender*: The reframed and expanded coverage theory renders feasible producing the deterministic latency distribution in a sampling-free manner, culminating in our practical discovery latency estimator Blender. This section briefly introduces the utility of Blender.

Input: The input of Blender mainly consists of configurations for broadcaster and scanner. For instance, the scanner configuration includes the scan interval T and scan window duration W ; The broadcaster configuration includes two branches: the determined parameterization with a finite broadcasting sequence Q or broadcast interval A . In addition, several stochastic factors can be involved, including the interference configuration represented by an average signal interference rate and the maximum random broadcasting delay (i.e., *adv_delay*), which is further discussed in Appendices B and C.

Modules in Blender: Blender follows a two-step procedure in producing the latency distribution:

- *Coverage Simulation*: From the perspective of the broadcaster, a sequence of broadcasting events is simulated to estimate the wall-clock time offset between the scanning sequence and the start of this broadcasting sequence. With δ_{bc} fixed as 0, the simulation results in a list of several intermediate discovery latency and their conditional probability.
- *Case Projection*: The latency from coverage simulation accounts for a fraction of all possible latency, which is the reason to annotate their result as *intermediate*. The range-entrance case projection (referred to as case projection), for example with fixed broadcast interval, projects case $\delta_{bc} = 0$ to all $\delta_{bc} \in [0, A)$, meanwhile producing the overall latency as the summation of a range-entrance latency (i.e., time from range-entrance to the first broadcasting event) and each intermediate latency. When considering stochastic factors, the case projection requires adaptations,

of which the explanation is complemented by the discussion of pragmatic adapters in the Appendix A.

The output is represented as a cumulative probability distribution of discovery latency in the complete domain, from which the P -percentile latency can be easily queried.

B. Common Interest Extraction

For each scan mode, the broadcast intervals near the valley or semi-valley area are defined as the *interest* of this scan mode. To search the global optimal broadcast intervals, we extract the common interest by finding the common broadcast intervals that achieve the locally minimized weight average discovery latency among n types of scan modes. First of all, we quantificationally give the definitions of the valley and semi-valley areas as below.

Definition 1 (Valley Area). According to [18], the local minimum P -percentile latency is computed as

$$l_{\min}^P = P \cdot \left\lceil \frac{T}{W} \right\rceil \cdot A \quad (8)$$

We then regard an A is within the valley area if its P -percentile discovery latency l_A^P equals to $l_{\min}^P$.

Definition 2 (Semi-valley Area). We regard an A is within the semi-valley area if its P -percentile latency meets $l_A^P \leq \alpha \cdot l_{\min}^P$, where α is a relaxation coefficient ($\alpha > 1$).

For each scan mode $s_i \in \mathbf{S}, i = 1, 2, \dots, n$, from the input of Common Interest Extraction, we can get the l_A^P , the P -percentile latency of any given A . By comparing l_A^P and $l_{\min}^P$ according to the definitions of valley area and semi-valley area, we can get the set of feasible broadcast intervals $\mathbf{B}_i$ that is within the valley or semi-valley areas. Then we compute the intersection $\mathbf{B}^*$ among n scan modes by $\mathbf{B}^* = \mathbf{B}_1 \cap \mathbf{B}_2 \cap \dots \cap \mathbf{B}_i \cap \dots \cap \mathbf{B}_n$.

Optimality Analysis: Through Common Interest Extraction, we obtain $\mathbf{B}^*$, a set of feasible broadcast intervals that are within the valley or semi-valley areas of all the n scan modes. According to (2) and (3), from $\mathbf{B}^*$ we can always find an interval that achieves the minimal weighted average discovery latency across all scan modes, called the *Single Broadcast Pattern*. We then formulate this problem as follows:

$$\min \hat{l} = \sum_{i=1}^n \omega_i \cdot l_{i,j}, \quad \text{s.t. } b_j \geq A_{\min}, \forall b_j \in \mathbf{B}^* \quad (9)$$

where $l_{i,j}$ is the worst-case discovery latency of scan mode s_i with the j^{th} feasible broadcast interval b_j ($b_j \in \mathbf{B}^*$). We further give an example of two types of scan modes s_1 and s_2 whose market shares are $\omega_1 = 30\%$ and $\omega_2 = 70\%$. Assume from Common Interest Extraction we obtain the intersection including 3 feasible broadcast intervals, i.e., $\mathbf{B}^* = \{b_1, b_2, b_3\} = \{1980 \text{ ms}, 2560 \text{ ms}, 4460 \text{ ms}\}$. Assume the power budget is $A_{\min} = 2000$ ms, then b_1 is discarded due to power constraint. Assume from Local Optima Estimation, we have $l_{1,2} = 5$ s, $l_{2,2} = 20$ s, $l_{1,3} = 8$ s and $l_{2,3} = 10$ s, then we compute the weighted average discovery latency of b_2 and b_3 as $\hat{l}_2 = 15.5$ s and $\hat{l}_3 = 9.4$ s, respectively. As a result, $b_3 = 4460$ ms becomes the optimal broadcast interval that is recommended by the ElastiCast algorithm.

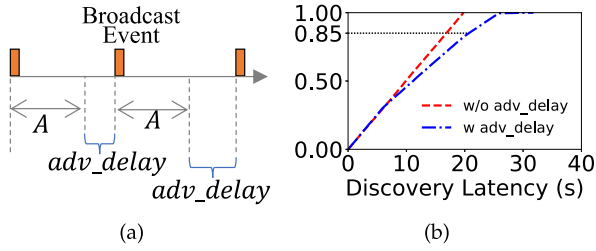


Fig. 14. (a) The adv_delay added to broadcast interval. (b) The adv_delay induces negative effects. $W = 60$ ms, $T = 600$ ms, and $A = 1980$ ms is within the valley area.

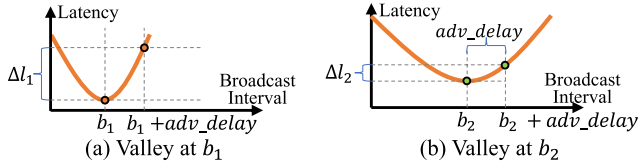


Fig. 15. Analysis on sensitivity to adv_delay .

However, simply setting a single optimal broadcast interval using the way depicted above might result in bias. This is because the implementations of the most modern BLE chips set a mandatory random advertising delay, i.e., adv_delay as illustrated in Fig. 14(a), before each broadcasting event [46]. adv_delay is not a constant and varies up to 10 ms. Thus, always adding a random time to the settled broadcast interval might result in a larger discovery latency than expected. Specifically, a broadcast interval within the valley area might be shifted to a non-valley area due to the mandatory and random adv_delay .

To explain this more clearly, we conduct experiments on how ElastiCast performs with and without adv_delay . Fig. 14(b) shows the results. Through simulation, we have known $A = 1980$ ms is within the valley area for the scan mode with $W = 60$ ms and $T = 600$ ms, i.e., it obtains the local minimum worst-case discovery latency ideally. However, in practical cases with adv_delay the tail latency (e.g., 85-percentile latency = 21 s) is larger than that in the ideal cases without adv_delay (e.g., 85-percentile latency = 16 s). Hence, adv_delay induces a negative effect for ElastiCast.

For a more practical ElastiCast, our next goal is to seek a way how to avoid the negative effect of adv_delay . Based on our long-term and comprehensive observations, we find that different broadcast intervals show different sensitivity to adv_delay . As illustrated in Fig. 15, assume the broadcast interval b_1 achieves the global minimal weighted average discovery latency among all scan modes. While compared with b_1 , the broadcast interval b_2 is less sensitive to the adv_delay , which obtains a lower latency bias (i.e., $\Delta l_2 < \Delta l_1$). We thus infer that the combination between b_2 and b_1 improves the adaptability in the context of the random adv_delay . Specifically, when adv_delay is large, the broadcast interval b_2 might achieve lower discovery latency than b_1 . This greatly motivates the Interval Multiplexing as we will elaborate next.

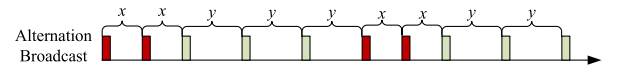


Fig. 16. An example of the Alternation Broadcast Pattern.

C. Interval Multiplexing

As discussed above, the Single Broadcast Pattern might fall short due to the existence of adv_delay . In this section, we explain how to step further toward the global optima by adopting the intermixed use of multiple broadcast intervals instead of the single one, thus we present the Alternation Broadcast Pattern as below.

Fig. 16 illustrates an example of the broadcasting event sequence when applying the Alternation Broadcast Pattern. In this pattern, two³ phases with intervals x and y ($x, y \in \mathbf{B}^*$) appear interchangeably. We define ϕ_x and ϕ_y as the repeated times of intervals x and y , respectively. For example in Fig. 16, we have $\phi_x = 2$ and $\phi_y = 3$. We define the *equivalent broadcast interval* (denoted by $\hat{A}$) as the average time between two consecutive broadcasting events. Then for the Alternation Broadcast Pattern, we have $\hat{A} = \frac{x\phi_x + y\phi_y}{\phi_x + \phi_y}$. When $\phi_x = 0$ or $\phi_y = 0$, the Alternation Broadcast Pattern falls back to the Single Broadcast Pattern.

Decision-Making: The decision-making is to select the best pattern and the corresponding parameter configuration from the Single Broadcast Pattern and the Alternation Broadcast Pattern. ElastiCast aims to achieve the minimized weighted average discovery latency within the power budget, therefore the constraint in (3) can be updated as $\hat{A} \geq \hat{A}_{\min}$, where $\hat{A}$ is the equivalent broadcast interval of broadcast pattern and $\hat{A}_{\min}$ is the minimum equivalent broadcast interval.

D. Discussion

This section discusses the provisioning principles for such parameters as the latency percentile (P), the relaxation coefficient (α), and the repeated times (ϕ_x and ϕ_y).

Latency Percentile: In OFN applications, the user experience is closely related to the success ratio (possibility) of finding the lost device, where the P -percentile discovery latency matters. Although P are customizable, to better fit the problem of achieving the highest success ratio within the valley or semi-valley area, it is recommended to compute P by $P = \min\{\frac{\text{latency_tolerance}}{l_{\min}^{\max}} \cdot 100, 100\}$, where latency_tolerance is the time a finder device spends within the BLE signal range in the walking scenario (see Fig. 5), and $l_{\min}^{\max} = \lceil \frac{T}{W} \rceil \cdot A$ is the local minimum worst-case latency [18].

Relaxation Coefficient: In general, setting a large relaxation coefficient α (e.g., $\alpha = 5$) expands the solution space of feasible broadcast intervals but also induces significant search overhead. On the other hand, setting a small α (e.g., $\alpha = 1.001$) shortens search time but may fail to extract the common interest. Hence,

³Generally, ElastiCast can multiplex as many feasible broadcast intervals as possible (i.e., ≥ 2) to search for the best pattern and parameter configuration. However, considering the exploding solution space and deployment hurdles, this paper only focuses on patterns consisting of two intervals and leaves the patterns consisting of more than two intervals for future work.

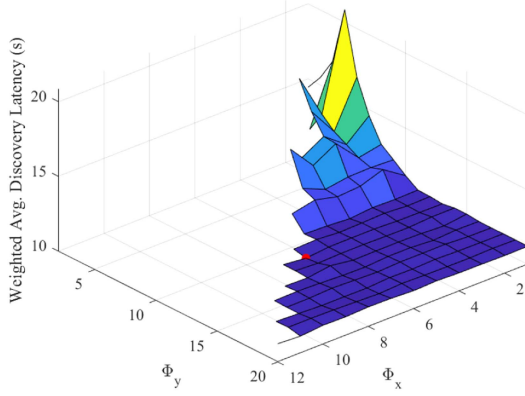


Fig. 17. An example of how to select the ϕ_x and ϕ_y combination. The settings of this experiment are the same as the one in Fig. 20. The red dot achieves the lowest weighted average discovery latency with the ϕ_x and ϕ_y combination (6, 12).

$\mathbf{B}^* = \emptyset$ means the relaxation factor α is set too small. In this paper, we increase α by 10% until it meets $\mathbf{B}^* \neq \emptyset$. This assures ElastiCast can always obtain a globally feasible solution with a bounded overhead.

Repeated Times: For the Alternation Broadcast Pattern, the repeated times ϕ_x and ϕ_y decide the phase duration proportion of the two broadcast intervals x and y . Note that ϕ_x and ϕ_y are not the input of ElastiCast but the output. The optimal ϕ_x and ϕ_y might vary with the inputs (e.g., scan modes) when the Alternation Broadcast Pattern is the preferred option. To select ϕ_x and ϕ_y , we straightforwardly search through combinations that result in a preferable percentile latency. As shown in Fig. 17, we search within the ranges $\phi_x \in [1, \phi_x^{\max}]$ and $\phi_y \in [1, \phi_y^{\max}]$, where $\phi_x^{\max}$ and $\phi_y^{\max}$ are calculated based on the scanning modes. For example, assuming there are n scan modes, $\phi_x^{\max}$ is computed as $\phi_x^{\max} = [\max\{\text{LCM}(x, T_i) \mid i = 1, 2, \dots, n\}] / x$. We then exhaustively search these combinations to select the optimal ϕ_x and ϕ_y that result in a preferable percentile latency.

Relationship among Alternation Broadcast Pattern, Coverage, and Latency Percentile: Interval multiplexing makes the CRT-based analysis unsuitable without a fixed broadcasting interval. Consequently, the latency estimation pivots to being governed by the extended Coverage Model. Importantly, the extended Coverage Model reveals the mechanism behind how dynamic broadcast interval overcomes the impact on P -percentile latency under *adv_delay*. While the model with dynamic broadcast interval shares the fundamental idea of *coverage* with fixed broadcast interval, we use the latter to simplify the following explanation. As implied in Section VI-A1, the latency distribution directly correlates with the scan window position distribution. In other words, the P -percentile latency will be $\Upsilon_i + b$, where after broadcasting event $\mathcal{A}_i$, the coverage status of segment $\mathcal{G}$ should be $\frac{|\mathcal{G}_{\text{covered}}|}{|\mathcal{G}|} \geq P\%$. Therefore, lowering P -percentile latency is equivalent to faster achieving a coverage of $P\%$ for $\mathcal{G}$. The fastest coverages for any P are achieved by *valley* broadcast intervals, stemming from the observation that there are no redundantly covered periods in $\mathcal{G}$. However, *adv_delay* can deteriorate this optimality. While the position of a broadcasting event becomes probabilistic, the originally deterministic period to be covered

in $\mathcal{G}$ becomes a distribution. Therefore, even if the broadcast interval belongs to *valley*, some of the samples from this distribution of periods could overlap with previously covered periods, slowing down the process towards $P\%$ coverage. When dynamic broadcast interval (e.g., interval multiplexing) is adopted, such a distribution of periods could be shifted along with a shifted broadcasting event, potentially reducing or even avoiding any redundant coverage caused by the periods in the distribution.

VII. EVALUATION OF BLENDER

Comparison on Percentile Latency: To validate Blender's basic functionality, the simulation results are compared with the measurement traces. Since one of the major targets of Blender is to provide the percentile latency, the comparison focuses on 20, 50, and 80-percentile latency, which lay through the most part of the distribution and preliminarily reflects the ability of the model-based output to fit the traces. We select 6 configurations for broadcasters, and 2 for scanners, and measure in 2 physical locations with different average packet loss rates.

Measuring the Effectiveness of Stochastic Factor Adaptors: Blender has equipped several modules to adapt stochastic factors, such as the random advertising delay and environmental interference, which introduces extra analysis and computation burden on the framework. To prove the significance of these excessive efforts, we compare Blender with a simulator constructed upon the *effective-scan-window* [16] model for ideal cases, denoted as TMC17. The RMSE between the distributions generated to the measurement traces are compared to show the ability to reflect pragmatic scenarios.

Measurement Setup: The testbed for measurement consists of two Android smartphones, which are installed with a controller application that manipulates BLE signal broadcasting/scanning. The application is developed based on an official example [47] with an additional module to measure the average packet loss rate, which is a necessary parameter for the simulation. The loss measurement module keeps one phone advertising the BLE packets N times and another phone continuously scans the advertising channel. In this way, the scanner can estimate the channel's packet loss rate by checking the percentage of successfully received advertising packets over N .

Evaluation results: Results are shown in Table III. For the parameters selected to produce the percentile latency, most of the results can act as a close estimation of the realistic situations. Although some of the 80-percentile latency shows a larger deviation, the trend of the real distribution is generally well-represented. To further represent the ability of the model to fit the realistic situation, the root-mean-square error (RMSE) is introduced as the metric to quantify the similarity between any two distributions. Fig. 18 shows the results with growing advertise interval under the *LOW_POWER* scan mode defined in Android BLE module [48]. When considering both stochastic factors, Blender results in much smaller RMSE values than the simulation that only considers the ideal scenario (i.e., TMC17) in an environment with on average 43% packet loss rate regardless of the selected advertising interval, which hence proves that the stochastic factors are worth handling. Conclusively, Blender is

TABLE III
PERCENTILE LATENCY BETWEEN SIMULATION (SIM) AND MEASUREMENT (MEA). (PARAMETER IN MILLISECONDS)

Parameter Type	Broadcaster A	Scanner $\frac{W}{T}$	Avg. Loss Rate	P20 L(Sim)	P20 L (Mea)	P50 L(Sim)	P50 L(Mea)	P80 L(Sim)	P80 L(Mea)
Randomized Broadcasting	1140			3.10s	4.28s	7.72s	8.59s	16.4s	15.1s
	3250			8.83s	9.59s	22.6s	20.4s	43.1s	39.7s
	6530	$\frac{512}{5120}$	26%	17.7s	15.6s	44.3s	43.4s	>50s	>50s
	1140			4.10s	4.38s	10.1s	11.9s	23.7s	27.5s
	3250			4.70s	2.78s	12.4s	13.7s	27.8s	31.1s
	6530	$\frac{1024}{4096}$	43%	9.17s	10.1s	24.9s	21.3s	>50s	48.3s
Flexible Duty-Cycle ⁴	1860×4								
	$+2140 \times 6$	$\frac{512}{5120}$	26%	5.48s	6.5s	15.1s	14.1s	33.6s	32.0s
	1280×2								
	$+3250 \times 4$			11.0s	9.27s	28.8s	26.8s	>50s	>50s
	$+4000 \times 1$		43%						
	860×5	$\frac{1024}{4096}$		3.22s	3.96s	7.45s	9.38s	21.3s	19.1s
	$+6530 \times 2$								

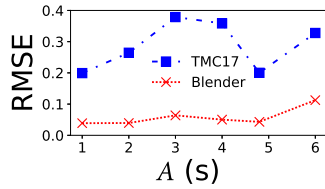


Fig. 18. RMSE of the distribution generated by Blender and ideal case simulator compared to the measurement traces.

capable of being applied in real-world scenarios and provides satisfactory outputs.

VIII. EVALUATION OF ELASTICAST

A. Experiment Setup

Inputs: A type of scan mode is in the form of W/T . For example, “1024 ms/4096 ms” refers to the scan mode with a scan window of 1024 ms and a scan interval of 4096 ms. The evaluation inputs are $s_i \in \mathbf{S}$ ($i = 1, 2, \dots, n$), ω_i , and $\hat{A}_{\min}$, where s_i is the i^{th} type of scan mode, ω_i is the market share of s_i , and $\hat{A}_{\min}$ represents the broadcaster’s power budget in the form of the minimum equivalent broadcast interval.

Parameters: We run tests for broadcast intervals that meet $A \in [A_{\text{left}}, A_{\text{right}}]$, where we recommend $A_{\text{left}} \geq 20$ ms and $A_{\text{right}} < \min\{10240 \text{ ms}, \text{latency_tolerance}\}$ according to the whole range of broadcast intervals allowed in BLE. In this paper, we set $A_{\text{left}} = 20$ ms and $A_{\text{right}} = 10240$ ms. The initial relaxation coefficient is set $\alpha = 1.2$.

Schemes: Let LOP (Local OPTima) be the scheme that sets the broadcast interval that achieves the tight duty-cycle-dependent bounds on discovery latency (i.e., $l_{\min}^P$ in (8)) for only one type of scan mode, i.e., LOP locally optimizes latency. Since no prior neighbor discovery parameter setting approaches can beat LOP in the case of homogeneous scan mode [18], we select LOP as

⁴Due to the limitation of Android BLE module, the flexible duty-cycle is currently measured from a random sampler imitating the detailed behavior of the BLE devices.

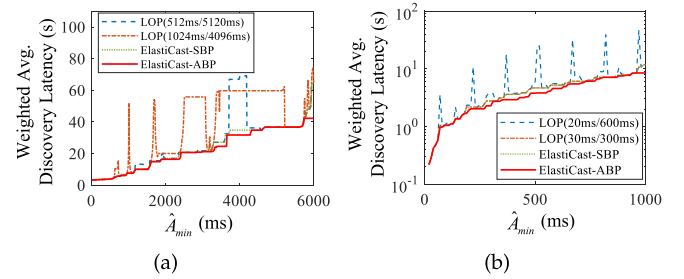


Fig. 19. Overall performance of ElastiCast. (a) Scan modes: 512 ms/5120 ms and 1024 ms/4096 ms. (b) Scan modes: 20 ms/600 ms and 30 ms/300 ms.

the baseline for ElastiCast evaluation in the case of scan mode diversity. *ElastiCast-SBP* refers to the Single Broadcast Pattern and *ElastiCast-ABP* refers to the Alternation Broadcast Pattern.

B. Performance Improvement of ElastiCast

In this experiment, we investigate two representative scenarios, where 512 ms/5120 ms (i.e., LOW_POWER) and 1024 ms/4096 ms (i.e., BALANCED) are two types of default scan modes supported by most Android phones [39], and 20 ms/600 ms and 30 ms/300 ms are two types of scan modes adopted by the applications of HarmonyOS (e.g., HiLink [44]) and iOS (e.g., Apple’s Find My [3]), respectively. The market shares of both scan modes are 50%. “LOP (20 ms/600 ms)” refers to the scheme that locally optimizes latency for the scan mode of 20 ms/600 ms, and so forth.

Fig. 19 shows the minimized weighted average discovery latency $\hat{l}$ (Y-axis) when applying each scheme within the power budget in the form of $\hat{A}_{\min}$ (X-axis). It is demonstrated that ElastiCast can always bound the neighbor discovery latency in the case of scan mode diversity. Specifically, Fig. 19(a) shows the performance with two scan modes 512 ms/5120 ms and 1024 ms/4096 ms. Both ElastiCast-SBP and ElastiCast-ABP outperform LOP in most cases. Particularly, compared to LOP(1024 ms/4096 ms), both ElastiCast-SBP and ElastiCast-ABP reduce the discovery latency by up to 50% in some

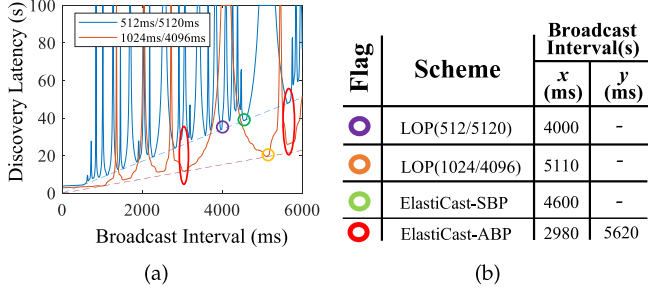


Fig. 20. (a) A case study on latency distribution when $\hat{A}_{\min} = 4000$ ms. (b) Examples of the selected broadcast interval(s) in different schemes.

cases. Fig. 19(b) shows the performance with two scan modes 20 ms/600 ms and 30 ms/300 ms. We can see that, compared to LOP(20 ms/600 ms), both ElastiCast-SBP and ElastiCast-ABP reduce the discovery latency by more than one order of magnitude (i.e., 90%) in some cases. Note that compared to ElastiCast-SBP, ElastiCast-ABP reduces the discovery latency by up to 40% in the scenario of 20 ms/600 ms and 30 ms/300 ms, but only gains a margin benefit in the scenario of 512 ms/5120 ms and 1024 ms/4096 ms. This can be attributed to the *adv_delay* that induces more negative effects in the scenario of 20 ms/600 ms and 30 ms/300 ms.

To explain the benefit of ElastiCast more clearly, we give a case study on latency distribution when $\hat{A}_{\min} = 4000$ ms. As shown in Fig. 20(a), we mark the selected/optimal broadcast interval(s) for each scheme with colored cycles. Fig. 20(b) further shows the exact values of broadcast intervals selected by corresponding schemes. For example, ElastiCast-SBP selects 4600 ms as the optimal broadcast interval, and ElastiCast-ABP selects both 2980 ms and 5620 ms ($\hat{A} = 4300$ ms) that appear interchangeably. Our decisions are then made by comparing the minimized discovery latency of all schemes.

C. Stability Analysis

How does the type of scan mode impact stability? First of all, we regard scan modes with different W or T as different types of scan modes even if they have the same duty cycle. This is because they have different valleys or semi-valley areas. For example, given $\hat{A}_{\min} = 4000$ ms, although the duty cycles are both 10%, the optimal broadcast interval of LOP(30 ms/300 ms) is 4030 ms, but the optimal broadcast interval of LOP(512 ms/5120 ms) is 4000 ms. As shown in the experiment results in Section VIII-B, ElastiCast shows stability in achieving global optima regardless of the types of scan modes.

How does the power budget impact stability? As shown in Fig. 19, the power budget impacts the decision-making significantly, however, ElastiCast still approximates a bounded and globally minimized latency.

How does the market share impact stability? We further vary the market shares of scan modes. For example, “3:7” represents that the market share of 512 ms/5120 ms is 30%, and the market share of 1024 ms/4096 ms is 70%. Fig. 21(a) shows that no matter how the market shares change, ElastiCast always approximates a bounded and globally minimized discovery latency.

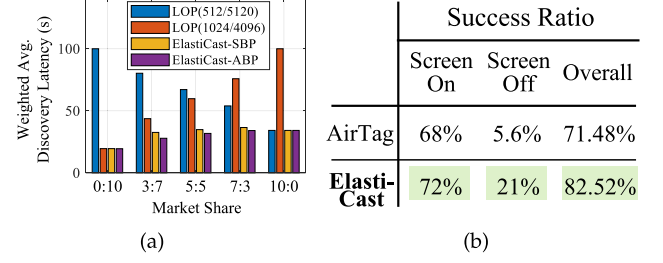


Fig. 21. (a) Performance under different market shares. (b) Performance comparison between AirTag and ElastiCast. (Screen On: 60 ms/600 ms, Screen Off: 60 ms/3000 ms).

D. Deployment Experience: HUAWEI Tag

ElastiCast has been deployed in Huawei’s commercial-off-the-shelf (COTS) BLE devices, called HUAWEI Tag [37]. HUAWEI Tag acts as the role of Lost Device (see Fig. 2) in the ecosystem of OFN. The Finder Devices are currently with two types of scan modes, i.e., 60 ms/600 ms and 60 ms/3000 ms, which refer to the scan modes applied when the screens of the Finder Devices are on and off, respectively. Based on our long-term market statistics, the market shares of the scan modes approximate 33% and 67%, respectively.

We compare ElastiCast with Apple’s AirTag which adopts a fixed broadcast interval of 2000 ms. We estimate the success ratio of finding the lost Tag with a latency tolerance of 14 seconds. Let N^+ be the number of tests that meet discovery latency of fewer than 14 seconds, and N^- otherwise. The success ratio is computed by $\frac{N^+}{N^+ + N^-}$. For a fair comparison, $\hat{A}_{\min}$ varies in a small range of [1950, 2050] ms. Fig. 21(b) shows the results. It is demonstrated that ElastiCast outperforms AirTag no matter whether the screen is on or off. In a case with three Finder Devices nearby, ElastiCast obtains an improvement of over 11% on the overall success ratio.

The lesson we have learned during deployment is that the scanner rarely strictly follows the instructions of parameter settings. For example, when we set the scan interval as 600 ms, the actual scan interval might be 605 ms with a small random bias, which we believe is attributed to the hardware/software task scheduling on the Finder Devices. Together with *adv_delay*, this may further impact the performance of the Single Broadcast Pattern. However, our evaluations demonstrate that the Alternation Broadcast Pattern can still compensate for the negative effect induced by the scanner side.

IX. CONCLUSION

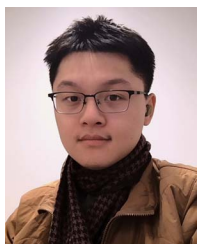
This paper examined the framework design and performance optimization in OFN. Our study identifies the unique features as well as the fundamental design challenges in OFN neighbor discovery. Our proposed ElastiCast has proven to be effective in achieving stable and low-latency neighbor discovery within the power budget in the case of scan mode diversity. The authors have provided public access to the code of Blender at <https://github.com/litonglab/blender-neighbor-discovery>.

REFERENCES

- [1] "Apple's find my feature," 2023. [Online]. Available: <https://www.apple.com/icloud/find-my/>
- [2] "A billion people now have iPhones," 2023. [Online]. Available: <https://www.aboveavalon.com/notes/2020/10/26/a-billion-iphone-users>
- [3] "AirTag Apple," 2023. [Online]. Available: <https://www.apple.com/sg/airtag/>
- [4] "Airtags are Apple's next billion dollar business," 2023. [Online]. Available: <https://www.forbes.com/sites/timbajarin/2021/04/20/airtags-are-apples-next-billion-dollar-business/?sh=75d236e65d18>
- [5] M. Weller, J. Classen, F. Ullrich, D. Waßmann, and E. Tews, "Lost and found: Stopping Bluetooth finders from leaking private information," in *Proc. 13th ACM Conf. Secur. Privacy Wireless Mobile Netw.*, 2020, pp. 184–194.
- [6] A. Heinrich, M. Stute, T. Kornhuber, and M. Hollick, "Who can find my devices? Security and privacy of Apple's crowd-sourced Bluetooth location tracking system," *Proc. Privacy Enhancing Technol.*, vol. 2021, no. 3, pp. 227–245, 2021.
- [7] T. Mayberry et al., "Who tracks the trackers? Circumventing Apple's anti-tracking alerts in the find my network," in *Proc. 20th Workshop Privacy Electron. Soc.*, 2021, pp. 181–186.
- [8] L. Tonetto, A. Carrara, A. Y. Ding, and J. Ott, "Where is my tag? Unveiling alternative uses of the Apple findmy service," in *Proc. IEEE 23rd Int. Symp. World Wireless Mobile Multimedia Netw.*, 2022, pp. 1–10.
- [9] Y. Zhao, K. Xu, H. Wang, B. Li, and R. Jia, "Stability-based analysis and defense against backdoor attacks on edge computing services," *IEEE Netw.*, vol. 35, no. 1, pp. 163–169, Jan./Feb. 2021.
- [10] S. Kamath and J. Lindh, "Measuring Bluetooth low energy power consumption," *Texas Instruments Application Note AN092*, 2010. [Online]. Available: <https://www.ti.com/lit/an/swra347a/swra347a.pdf>
- [11] J. Liu, C. Chen, and Y. Ma, "Modeling neighbor discovery in Bluetooth low energy networks," *IEEE Commun. Lett.*, vol. 16, no. 9, pp. 1439–1441, Jan./Feb. 2012.
- [12] P. Kindt, D. Yunge, R. Diemer, and S. Chakraborty, "Precise energy modeling for the Bluetooth low energy protocol," 2014, *arXiv:1403.2919*.
- [13] H. Lee, D. Ok, J. Han, I. Hwang, and K. Kim, "Performance anomaly of neighbor discovery in Bluetooth low energy," in *Proc. IEEE Int. Conf. Consum. Electron.*, 2016, pp. 341–342.
- [14] K. Cho, G. Park, W. Cho, J. Seo, and K. Han, "Performance analysis of device discovery of Bluetooth low energy (BLE) networks," *Comput. Commun.*, vol. 81, pp. 72–85, 2016.
- [15] W. S. Jeon, M. H. Dwijaksara, and D. G. Jeong, "Performance analysis of neighbor discovery process in Bluetooth low-energy networks," *IEEE Trans. Veh. Technol.*, vol. 66, no. 2, pp. 1865–1871, 2016.
- [16] P. H. Kindt, M. Saur, M. Balszun, and S. Chakraborty, "Neighbor discovery latency in BLE-Like protocols," *IEEE Trans. Mobile Comput.*, vol. 17, no. 3, pp. 617–631, Mar. 2018.
- [17] A. Liendo, D. Morche, R. Guizzetti, and F. Rousseau, "BLE parameter optimization for IoT applications," in *Proc. IEEE Int. Conf. Commun.*, 2018, pp. 1–7.
- [18] P. H. Kindt and S. Chakraborty, "On optimal neighbor discovery," in *Proc. ACM Special Int. Group Data Commun.*, 2019, pp. 441–457.
- [19] "AirTag analysis," 2023. [Online]. Available: <https://adamcatley.com/AirTag.html>
- [20] Y. Ding, T. Li, J. Liang, and D. Wang, "Blender: Toward practical simulation framework for BLE neighbor discovery," in *Proc. 25th Int. ACM Conf. Model. Anal. Simul. Wireless Mobile Syst.*, 2022, pp. 103–110.
- [21] S. Vasudevan, J. Kurose, and D. Towsley, "On neighbor discovery in wireless networks with directional antennas," in *Proc. IEEE 24th Annu. Joint Conf. IEEE Comput. Commun. Societies.*, 2005, pp. 2502–2512.
- [22] R. Zheng, J. C. Hou, and L. Sha, "Optimal block design for asynchronous wake-up schedules and its applications in multihop wireless networks," *IEEE Trans. Mobile Comput.*, vol. 5, no. 9, pp. 1228–1241, Sep. 2006.
- [23] Z. Zhang and B. Li, "Neighbor discovery in mobile ad hoc self-configuring networks with directional antennas: Algorithms and comparisons," *IEEE Trans. Mobile Comput.*, vol. 7, no. 5, pp. 1540–1549, May 2008.
- [24] S. Vasudevan, D. Towsley, D. Goeckel, and R. Khalili, "Neighbor discovery in wireless networks and the coupon collector's problem," in *Proc. 15th Annu. Int. Conf. Mobile Comput. Netw.*, 2009, pp. 181–192.
- [25] N. Karowski, A. C. Viana, and A. Wolisz, "Optimized asynchronous multi-channel neighbor discovery," in *Proc. IEEE INFOCOM*, 2011, pp. 536–540.
- [26] S. Vasudevan, M. Adler, D. Goeckel, and D. Towsley, "Efficient algorithms for neighbor discovery in wireless networks," *IEEE/ACM Trans. Netw.*, vol. 21, no. 1, pp. 69–83, Feb. 2013.
- [27] W. Sun, Z. Yang, K. Wang, and Y. Liu, "Hello: A generic flexible protocol for neighbor discovery," in *Proc. IEEE Conf. Comput. Commun.*, 2014, pp. 540–548.
- [28] T. Meng, F. Wu, and G. Chen, "On designing neighbor discovery protocols: A code-based approach," in *Proc. IEEE Conf. Comput. Commun.*, 2014, pp. 1689–1697.
- [29] W. Sun, Z. Yang, X. Zhang, and Y. Liu, "Energy-efficient neighbor discovery in mobile ad hoc and wireless sensor networks: A survey," *IEEE Commun. Surv. Tut.*, vol. 16, no. 3, pp. 1448–1459, Third Quarter 2014.
- [30] L. Chen, R. Fan, K. Bian, M. Gerla, T. Wang, and X. Li, "On heterogeneous neighbor discovery in wireless sensor networks," in *Proc. IEEE Conf. Comput. Commun.*, 2015, pp. 693–701.
- [31] Y. Qiu, S. Li, X. Xu, and Z. Li, "Talk more listen less: Energy-efficient neighbor discovery in wireless sensor networks," in *Proc. 35th Annu. IEEE Int. Conf. Comput. Commun.*, 2016, pp. 1–9.
- [32] B. Luo, J. Xu, and Z. Sun, "Neighbor discovery latency in Bluetooth low energy networks," *Wireless Netw.*, vol. 26, pp. 1773–1780, 2020.
- [33] G. Shan and B.-H. Roh, "Performance model for advanced neighbor discovery process in Bluetooth low energy 5.0-enabled Internet of Things networks," *IEEE Trans. Ind. Electron.*, vol. 67, no. 12, pp. 10965–10974, Dec. 2020.
- [34] W. Bai et al., "Euclidean-division-based low-complexity precise analytical approach of BLE-like neighbor discovery latency," *IEEE Internet Things J.*, vol. 10, no. 5, pp. 4184–4201, Mar. 2023.
- [35] A. Antipa, D. Brown, A. Menezes, R. Struik, and S. Vanstone, "Validation of elliptic curve public keys," in *Public Key Cryptography*. Berlin, Germany: Springer, 2002, pp. 211–223.
- [36] K. Geissdoerfer and M. Zimmerling, "Bootstrapping battery-free wireless networks: Efficient neighbor discovery and synchronization in the face of intermittency," in *Proc. USENIX Conf. Networked Syst. Des. Implementation*, 2021, pp. 439–455.
- [37] "Huawei tag," 2023. [Online]. Available: <https://consumer.huawei.com/cn/accessories/tag/>
- [38] "What is the range of Bluetooth and how can it be extended," 2023. [Online]. Available: <https://www.scienceabc.com/innovation/what-is-the-range-of-bluetooth-and-how-can-it-be-extended.html>
- [39] "Android BLE scan settings APIs," 2023. [Online]. Available: <https://developer.android.com/reference/android/bluetooth/le/ScanSettings>
- [40] "Exposure notification," 2020. [Online]. Available: <https://www.google.com/covid19/exposurenotifications>
- [41] "Huawei contact shield," 2020. [Online]. Available: <https://developer.huawei.com/consumer/en/doc/development/system-Guides/contactsshield-introduction-0000001057494465>
- [42] "AirTag Apple," 2021. [Online]. Available: <https://www.apple.com/sg/airtag/>
- [43] J. Liu, C. Chen, and Y. Ma, "Modeling and performance analysis of device discovery in Bluetooth low energy networks," in *Proc. IEEE Glob. Commun. Conf.*, 2012, pp. 1538–1543.
- [44] "HUAWEI Huawei," 2023. [Online]. Available: <https://iot.hilink.huawei.com/>
- [45] T. Li et al., "TACK: Improving wireless transport performance by taming acknowledgments," in *Proc. Annu. Conf. ACM Special Int. Group Data Commun. Appl. Techno. Architectures Protoc. Comput. Commun.*, 2020, pp. 15–30.
- [46] "Bluetooth specifications," 2023. [Online]. Available: <https://www.bluetooth.com/>
- [47] "Android samples," 2021. [Online]. Available: <https://github.com/android/connectivity-samples/tree/main/BluetoothAdvertisements>
- [48] "Android BLE documentation," 2022. [Online]. Available: <https://developer.android.com/reference/android/bluetooth/le/ScanSettings>



Tong Li (Member, IEEE) received the PhD degree from Tsinghua University, Beijing, China, in 2017. He held a visiting scholar with the School of Computer Science and Electronic Engineering of University of Essex, U.K., in 2014 and 2016. He is currently an associate professor with the Renmin University of China. His research interests include network protocols, the Internet of Things, and distributed systems. He was a former chief engineer with Huawei, and was the 2021 Huawei's Top Ten Inventions Award and the 2024 ACM SIGCOMM China Rising Star Award.



Yukuan Ding received the MSc degree in information technology from the Hong Kong University of Science and Technology, Hong Kong SAR, in 2023. He is currently working toward the PhD degree with the Delft University of Technology. His research interests include long-range wireless networks, Artificial Intelligence of Things, and distributed systems.



Tian Pan received the PhD degree from the Department of Computer Science and Technology, Tsinghua University, in 2015. He is currently an associate professor with the Beijing University of Posts and Telecommunications. His primary research interests include cloud data center networks, programmable data planes, and satellite networks.



Jiaxin Liang received the PhD degree in information engineering from the Chinese University of Hong Kong, in 2021. He is currently working toward the principle R&D engineer with Huawei in Computer Network and Protocol Research Laboratory. His research interests include network protocols, low-energy transmission, the Internet of Things (IoT), multiple access networks, and software-defined radios.



Dan Wang (Senior Member, IEEE) received the BSc degree in computer science from Peking University, Beijing, the MSc degree in computer science from Case Western Reserve University, Cleveland, OH, and the PhD degree from Simon Fraser University, Vancouver, Canada. He is a professor with the Department of Computing, The Hong Kong Polytechnic University. His research interests include sensor networks, Internet architecture, and protocols. Recently, he has been working on Cloud Computing and Big Data, to support interdisciplinary domains in Smart Cities.



Kai Zheng (Senior Member, IEEE) received the PhD degree in computer science from Tsinghua University, Beijing, China, in 2006. He is the director of the Computer Network and Protocol Research Lab, Huawei Technologies. His research interests include covered architectures and protocols for the next generation networks (e.g. 5G/IoT Networks, cloud-oriented data center networks, RDMA networks, and real-time multimedia networks, etc).



Ke Xu (Fellow, IEEE) received the PhD degree from Tsinghua University, Beijing, China. He is currently a full professor with the Department of Computer Science and Technology, Tsinghua University. He has published more than 200 technical articles in the research areas of next-generation internet, blockchain systems, and network security. He has won the IWQoS 2024 Best Paper Award and the Distinguished Paper Award at USENIX Security 2023 and 2024.



Xu Zhang received the PhD degree in computer science from the Department of Computer Science and Technology, Tsinghua University, China, in 2017. He is with the School of Electronic Science and Engineering, Nanjing University, China. His research interests include AI for computing and networking, multimedia networks, and Internet of Things. He was a recipient of the EU Marie Skłodowska-Curie Individual Fellowship and a co-recipient of the 2019 IEEE Broadcast Technology Society Best Paper Award.